UNIVERSIDAD AUTÓNOMA DEL CARIBE

MANUAL DE UTILIZACIÓN E IMPLEMENTACIÓN DEL SOFTWARE AUTOTRIGGER

Mateo David Castro Villegas
Geymar David Chung Soto
Duván Cohen Pérez
Carlos Gabriel Díaz Sáenz
Jean Pierre Coll Vásquez
Kelvin De Jesús Beleño Sáenz
Saúl Antonio Pérez Pérez

ÍNDICE

Introducción
1.1 ¿Qué es autotrigger?
1.2 Arquitectura General del Sistema
Descripción General del Código y su Funcionamiento
2.1 Visión General: El Corazón del Lanzador
2.2 Definiciones y Constantes Globales
2.3 Variables de Estado
2.4 La Función setup(): Configuración Inicial
2.5 La Función loop(): El Bucle Operativo Constante
2.6 procesarComando(): El Intérprete de Órdenes
2.7 moverBase(): Control de la Rotación Horizontal
2.8 moverServoY(): Control de la Elevación Vertical
2.9 iniciarDisparo(): El Mecanismo de Activación
2.10 Funciones Auxiliares de Sistema
frenarMotor(): Detener la Base
enviarConfirmacion(): Feedback de Comando
debugEstado(): Monitoreo Detallado (Solo en DEBUG_MODE)
2.11 Funciones de Prueba y Restablecimiento
pruebaRele(): Verificación del Relé
resetPosiciones(): Reinicio del Sistema
Requerimientos del Sistema
3.1 Hardware
3.2 Software
Primeros Pasos: Configuración y Carga del Firmware
4.1 Preparación del Entorno (Arduino IDE)
4.2 Carga del Código al ESP32
Modo de Operación y Comandos
5.1 Conexión Serial
5.2 Estructura de los Comandos
5.3 Comandos Especiales
Consideraciones Adicionales

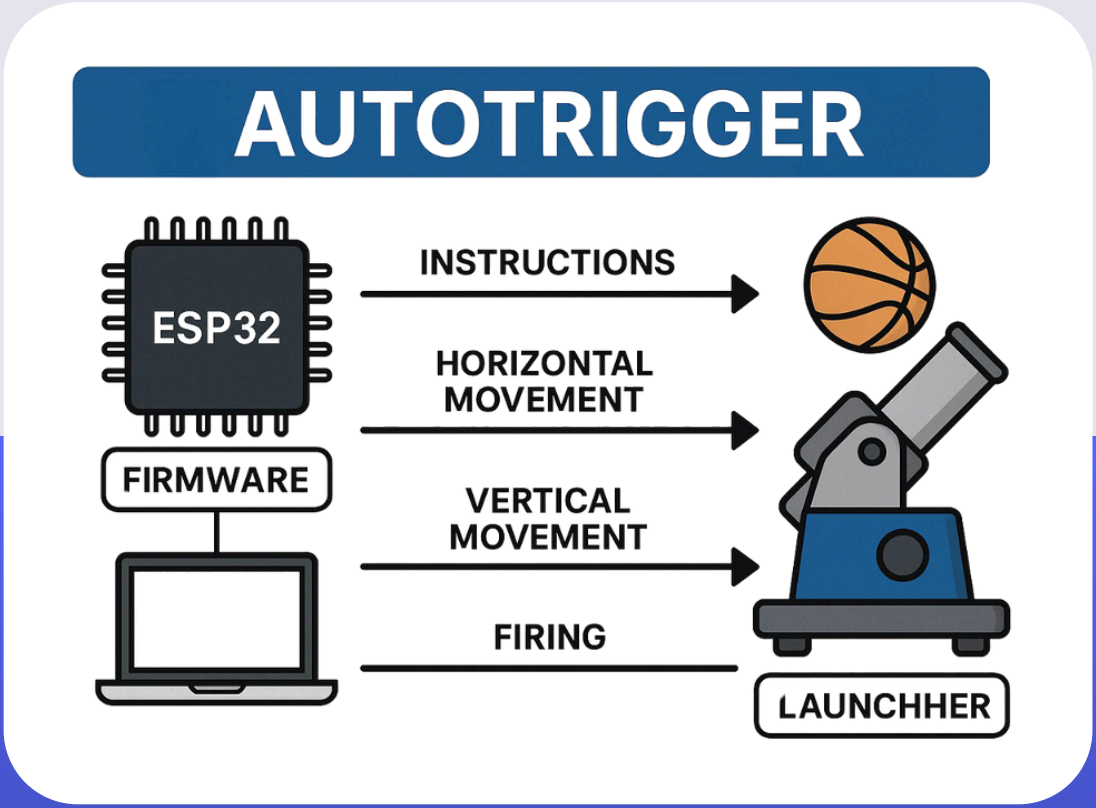
1. Introducción

1.1 ¿Qué es autotrigger?

El firmware autotrigger es el "cerebro" que reside en nuestro microcontrolador ESP32. Su misión es simple pero crucial: recibir e interpretar las instrucciones que le llegan desde un software externo (como LaunchassistPy) y, basándose en ellas, controlar los movimientos precisos de nuestro sistema de lanzamiento de balones. Se encarga tanto del apuntado horizontal como del vertical, y por supuesto, de la activación del disparo.

1.2 Arquitectura General del Sistema

autotrigger es la pieza intermedia de nuestro sistema. Actúa como un traductor entre las órdenes de alto nivel de un software de visión (que detecta las manos) y los actuadores físicos del lanzador. La comunicación es serial, lo que lo hace robusto y directo.



2. Descripción General del Código y su Funcionamiento

2.1 Visión General: El Corazón del Lanzador
El código autotrigger es un programa en C++ diseñado para el microcontrolador ESP32, utilizando el entorno de Arduino IDE. Su propósito principal es convertir comandos seriales simples en acciones físicas complejas: mover motores y servos con precisión y activar un relé de disparo. Está diseñado para ser robusto, con validaciones de rangos y un sistema de depuración integrado para facilitar el desarrollo y mantenimiento. En esencia, autotrigger es el intérprete y el ejecutor de las intenciones de apuntado y disparo que provienen de un sistema de control de nivel superior.

2.2 Definiciones y Constantes Globales

Estas líneas al inicio del código establecen los mapeos de pines y valores fijos que no cambiarán durante la ejecución del programa. Son cruciales para la interacción con el hardware.

2.2 DEFINICIONES Y CONSTANTES GLOBALES

Estas líneas al inicio del código establecen los mapeos de pines y valores fijos que no cambiarán durante la ejecución del programa. Son cruciales para la interacción con el hardware.

```
#INCLUDE <ESP32SERVO.H> // NECESARIA PARA CONTROLAR LOS SERVOMOTORES EN ESP32

// PINES DEL MOTOR DC
#DEFINE MOTOR_IN1 25 // PIN PARA CONTROL DE DIRECCIÓN 1 DEL PUENTE H
#DEFINE MOTOR_IN2 26 // PIN PARA CONTROL DE DIRECCIÓN 2 DEL PUENTE H
#DEFINE MOTOR_ENA 27 // PIN PARA HABILITAR Y CONTROLAR LA VELOCIDAD (PWM) DEL MOTOR

// PINES DE CONTROL
#DEFINE SERVO_Y_PIN 13 // PIN AL QUE SE CONECTA EL SERVOMOTOR DEL EJE Y (ELEVACIÓN)
#DEFINE DISPARO_PIN 14 // PIN AL QUE SE CONECTA EL MÓDULO DE RELÉ PARA EL DISPARO

// LÍMITES SERVO Y
#DEFINE SERVO_Y_MIN 30 // ÁNGULO MÍNIMO AL QUE PUEDE MOVERSE EL SERVO Y
#DEFINE SERVO_Y_MAX 150 // ÁNGULO MÁXIMO AL QUE PUEDE MOVERSE EL SERVO Y
#DEFINE SERVO_Y_CENTER 90 // POSICIÓN CENTRAL INICIAL PARA EL SERVO Y

// TIEMPOS CALIBRADOS (MS POR GRADO) PARA EL MOTOR DE LA BASE
// ESTOS VALORES SON CRÍTICOS Y DEPENDEN DE LA CALIBRACIÓN DE TU MOTOR Y MECÁNICA.

CONST FLOAT TIEMPO_POR_GRADO_HORARIO = 5.6; // TIEMPO EN MS PARA GIRAR 1 GRADO EN
SENTIDO HORARIO (0 A 180)
CONST FLOAT TIEMPO_POR_GRADO_ANTIHORARIO = 5.3; // TIEMPO EN MS PARA GIRAR 1 GRADO
EN SENTIDO ANTIHORARIO (180 A 0)
```

- **#include <ESP32Servo.h>:** Incluye la librería necesaria para manejar los servomotores de forma específica en el ESP32, aprovechando sus capacidades de PWM.
- **#define MOTOR_IN1, MOTOR_IN2, MOTOR_ENA:** Asignan nombres simbólicos (constantes de preprocesador) a los números de los pines GPIO del ESP32 que se utilizarán para controlar el puente H del motor de corriente continua (DC).
- **#define SERVO_Y_PIN, DISPARO_PIN:** Similar a los anteriores, definen los pines para el servomotor de elevación y el relé de disparo.
- **#define SERVO_Y_MIN, SERVO_Y_MAX, SERVO_Y_CENTER:** Establecen los límites operativos seguros para el servomotor del eje Y y su posición de inicio/centro. Es fundamental que estos valores se ajusten a los límites físicos de tu mecanismo para evitar daños.
- **const float TIEMPO_POR_GRADO_HORARIO,TIEMPO_POR_GRADO_ANTIHORARIO:** Son constantes de punto flotante que almacenan los valores de calibración. Estos números representan la cantidad de milisegundos que el motor de la base necesita estar activo para girar un solo grado en la dirección especificada. Son el resultado de pruebas empíricas para lograr un movimiento horizontal preciso

2.3 VARIABLES DE ESTADO

Estas variables son declaradas globalmente para que todas las funciones del programa puedan acceder a ellas y modificarlas. Mantienen el registro del estado actual del lanzador.

```
SERVO SERVOY; // OBJETO DE LA LIBRERÍA ESP32SERVO PARA CONTROLAR EL SERVOMOTOR DEL EJE Y

// VARIABLES DE ESTADO DEL SISTEMA
INT POSICIONBASE = 90; // ALMACENA LA POSICIÓN ACTUAL ESTIMADA DE LA BASE DEL LANZADOR (0-180 GRADOS).
CONST INT VELOCIDADMOTOR = 255; // VELOCIDAD DEL MOTOR DE LA BASE (0-255, SIENDO 255 LA MÁXIMA VELOCIDAD PWM).

// VARIABLES PARA EL CONTROL DEL DISPARO
UNSIGNED LONG TIEMPOINICIODISPARO = 0; // ALMACENA EL VALOR DE MILLIS() CUANDO SE INICIA UN DISPARO.
CONST UNSIGNED LONG DURACION_PULSO_DISPARO = 500; // DURACIÓN DEL PULSO DE ACTIVACIÓN DEL RELÉ EN MILISEGUNDOS (0.5 SEGUNDOS).
BOOL DISPAROACTIVO = FALSE; // BANDERA QUE INDICA SI UN CICLO DE DISPARO ESTÁ ACTUALMENTE EN CURSO.

// VARIABLE PARA EL MODO DE DEPURACIÓN
CONST BOOL DEBUG_MODE = TRUE; // SI ES 'TRUE', HABILITA LA IMPRESIÓN DE MENSAJES DETALLADOS POR SERIAL. ÚTIL PARA EL DESARROLLO Y DIAGNÓSTICO.
)
```

- **Servo servoY;:** Se crea una instancia del objeto Servo que será utilizada para interactuar con el servomotor conectado al pin SERVO_Y_PIN.
- **int posicionBase = 90;:** Representa la creencia del firmware sobre dónde está ubicada la base horizontalmente. Se inicializa en 90 grados, asumiendo una posición central.
- **const int velocidadMotor = 255;:** Define la velocidad constante a la que operará el motor de la base. Se usa con la función analogWrite().
- **unsigned long tiempoInicioDisparo = 0;:** Una variable de tiempo sin signo largo, diseñada para almacenar el valor retornado por millis() (tiempo transcurrido desde el inicio del programa en ms) cuando el relé de disparo se activa.
- **const unsigned long DURACION_PULSO_DISPARO = 500;:** Define la duración exacta en milisegundos que el relé de disparo permanecerá activo.
- **bool disparoActivo = false;:** Una bandera booleana (true o false) que indica si el relé de disparo está actualmente activado. Esto evita que se lancen múltiples pulsos si la señal de disparo se mantiene por más tiempo del necesario.
- **const bool DEBUG_MODE = true;:** Una constante booleana que controla si los mensajes de depuración se imprimen por el puerto serial. Cuando está en true, el Monitor Serial mostrará información detallada del funcionamiento interno, lo cual es invaluable durante el desarrollo y la solución de problemas. Para una operación "silenciosa", debe establecerse en false.

2.4 La Función setup(): Configuración Inicial

Esta función es lo primero que se ejecuta cuando el ESP32 se enciende o se reinicia. Es donde se realizan todas las configuraciones iniciales del hardware y la comunicación.

```
VOID SETUP() {
 SERIAL.BEGIN(115200); // INICIALIZA LA COMUNICACIÓN SERIAL A 115200 BAUDIOS. FUNDAMENTAL PARA RECIBIR COMANDOS.

 // CONFIGURACIÓN DE PINES DEL MOTOR DC COMO OUTPUT
 PINMODE(MOTOR_IN1, OUTPUT);
 PINMODE(MOTOR_IN2, OUTPUT);
 PINMODE(MOTOR_ENA, OUTPUT);

 // CONFIGURACIÓN DEL PIN DE DISPARO COMO OUTPUT Y ASEGURARLO LOW (APAGADO) AL INICIO
 PINMODE(DISPARO_PIN, OUTPUT);
  DIGITALWRITE(DISPARO_PIN, LOW); // ESTO ES UNA MEDIDA DE SEGURIDAD PARA EVITAR DISPAROS ACCIDENTALES AL ENCENDER.

 // CONFIGURACIÓN DEL SERVOMOTOR DEL EJE Y
  ESP32PWM::ALLOCATETIMER(0); // ASIGNA UNO DE LOS TEMPORIZADORES PWM DEL ESP32 PARA EL CONTROL DEL SERVO.
 SERVOY.SETPERIODHERTZ(50); // ESTABLECE LA FRECUENCIA DE LA SEÑAL PWM PARA EL SERVO EN 50 HZ.
 SERVOY.ATTACH(SERVO_Y_PIN, 500, 2400); // ASOCIA EL OBJETO SERVOY AL PIN SERVO_Y_PIN Y DEFINE LOS LÍMITES DE PULSO PWM (EN MICROSEGUNDOS) PARA EL RANGO COMPLETO DEL SERVO. ESTOS VALORES SON TÍPICOS PARA LA MAYORÍA DE LOS SERVOS.
  SERVOY.WRITE(SERVO_Y_CENTER); // MUEVE EL SERVOMOTOR A LA POSICIÓN CENTRAL DEFINIDA (90 GRADOS).

 FRENARMOTOR(); // LLAMA A LA FUNCIÓN PARA ASEGURAR QUE EL MOTOR DE LA BASE ESTÉ DETENIDO AL INICIO.

 IF(DEBUG_MODE) { // SI EL MODO DEPURACIÓN ESTÁ ACTIVO, IMPRIME MENSAJES DE INICIO ÚTILES.
SERIAL.PRINTLN("SISTEMA INICIALIZADO");
SERIAL.PRINTLN("MODO DEPURACIÓN ACTIVADO");
SERIAL.PRINTLN("FORMATO DE COMANDO: BXYYFZ");
SERIAL.PRINTLN("EJEMPLO: B90Y75F1");
 }
}
```

- **Serial.begin(115200);:** Inicia la comunicación serial a una velocidad de 115200 baudios. Es crucial que el dispositivo que se comunica con el ESP32 (e.g., Raspberry Pi) utilice la misma velocidad para evitar corrupción de datos.
- **pinMode(...):** Configura los pines definidos para el motor y el relé como OUTPUT. Esto significa que el ESP32 enviará señales a través de estos pines para controlar los componentes.
- **digitalWrite(DISPARO_PIN, LOW);:** Una medida de seguridad importante. Asegura que el relé de disparo esté apagado cuando el sistema se enciende, previniendo lanzamientos involuntarios.
- **Configuración del Servo (ESP32PWM::allocateTimer(0), servoY.setPeriodHertz(50), servoY.attach(...), servoY.write(SERVO_Y_CENTER)):** Estas líneas inicializan el servomotor. Asignan un temporizador PWM específico del ESP32, establecen la frecuencia de la señal de control del servo y lo asocian al pin SERVO_Y_PIN con sus límites de pulso. Finalmente, lo mueven a su posición central.
- **frenarMotor();:** Se llama para asegurar que el motor de la base esté completamente detenido cuando el sistema arranca.
- **Mensajes de Depuración:** Si DEBUG_MODE está activado, se imprimen mensajes útiles al Monitor Serial para confirmar la inicialización y recordar el formato de los comandos.

2.6 ProcesarComando(): El Intérprete de Órdenes

Esta función es la encargada de analizar las cadenas de texto recibidas por serial y extraer los parámetros necesarios para controlar el lanzador. Es el "parseador" de comandos.

```
VOID PROCESARCOMANDO(STRING COMANDO) {
 // --- COMANDOS ESPECIALES ---
 // SI EL COMANDO ES EXACTAMENTE "TESTFIRE", LLAMA A LA FUNCIÓN DE PRUEBA DE RELÉ.
 IF (COMANDO == "TESTFIRE") {
PRUEBARELE();
RETURN; // SALE DE LA FUNCIÓN DESPUÉS DE EJECUTAR EL COMANDO ESPECIAL.
 }

 // SI EL COMANDO ES EXACTAMENTE "RESET", LLAMA A LA FUNCIÓN DE RESETEO.
 IF (COMANDO == "RESET") {
RESETPOSICIONES();
RETURN; // SALE DE LA FUNCIÓN.
 }

 // --- PROCESAMIENTO DE COMANDO NORMAL BXYYFZ ---
 // BUSCA LA POSICIÓN DE LAS LETRAS CLAVE EN LA CADENA DE COMANDO.
 INT BASEIDX = COMANDO.INDEXOF('B');
 INT YIDX = COMANDO.INDEXOF('Y');
 INT FIDX = COMANDO.INDEXOF('F');

 // VERIFICA QUE LAS LETRAS CLAVE EXISTAN Y ESTÉN EN EL ORDEN CORRECTO (B < Y < F).
 IF (BASEIDX != -1 && YIDX != -1 && FIDX != -1 &&
   BASEIDX < YIDX && YIDX < FIDX) {
  // EXTRAE LOS VALORES NUMÉRICOS DE LA CADENA Y LOS CONVIERTE A ENTEROS.
INT BASEANGLE = COMANDO.SUBSTRING(BASEIDX + 1, YIDX).TOINT(); // EXTRAE EL NÚMERO DESPUÉS DE 'B' Y ANTES
DE 'Y'.
INT YANGLE = COMANDO.SUBSTRING(YIDX + 1, FIDX).TOINT();   // EXTRAE EL NÚMERO DESPUÉS DE 'Y' Y ANTES DE 'F'.
BOOL FIRE = (COMANDO.SUBSTRING(FIDX + 1).TOINT() == 1);   // EXTRAE EL NÚMERO DESPUÉS DE 'F' Y LO CONVIERTE
A BOOLEANO (1=TRUE, 0=FALSE).

  // --- VALIDACIÓN Y AJUSTE DE ÁNGULOS ---
     // CONSTRAIN() ASEGURA QUE LOS ÁNGULOS ESTÉN DENTRO DE SUS LÍMITES DEFINIDOS, EVITANDO
MOVIMIENTOS FUERA DE RANGO.
  BASEANGLE = CONSTRAIN(BASEANGLE, 0, 180);
  YANGLE = CONSTRAIN(YANGLE, SERVO_Y_MIN, SERVO_Y_MAX);

  // --- MOVER BASE SI HAY CAMBIO SIGNIFICATIVO ---
  // SOLO MUEVE LA BASE SI EL ÁNGULO OBJETIVO DIFIERE DEL ACTUAL EN MÁS DE 1 GRADO PARA EVITAR MICRO-
MOVIMIENTOS.
IF (ABS(BASEANGLE - POSICIONBASE) > 1) {
MOVERBASE(BASEANGLE); // LLAMA A LA FUNCIÓN PARA MOVER LA BASE.
  } ELSE {
FRENARMOTOR(); // SI NO HAY CAMBIO RELEVANTE, ASEGURA QUE EL MOTOR DE LA BASE ESTÉ PARADO.
  }

  // --- MOVER SERVO Y ---
MOVERSERVOY(YANGLE); // LLAMA A LA FUNCIÓN PARA MOVER EL SERVO DE ELEVACIÓN.

  // --- MANEJAR DISPARO ---
  // SI LA SEÑAL DE DISPARO ES 'TRUE' (1) Y NO HAY UN DISPARO EN CURSO.
IF (FIRE && !DISPAROACTIVO) {
INICIARDISPARO(); // LLAMA A LA FUNCIÓN PARA INICIAR EL DISPARO.
  }

 // --- CONFIRMACIÓN DE COMANDO ---
ENVIARCONFIRMACION(BASEANGLE, YANGLE, FIRE); // ENVÍA UNA RESPUESTA AL DISPOSITIVO QUE ENVIÓ
EL COMANDO.

 } ELSE IF(DEBUG_MODE) { // SI EL FORMATO DEL COMANDO NO ES VÁLIDO Y EL DEBUG_MODE ESTÁ ACTIVO.
SERIAL.PRINTLN("ERROR: FORMATO DE COMANDO INVÁLIDO");
SERIAL.PRINTLN("EJEMPLO CORRECTO: B90Y75F1");
 }
}
```

- **Comandos Especiales (TESTFIRE, RESET):** La función primero verifica si el comando es uno de estos comandos especiales y los ejecuta directamente, saliendo de la función con return.
- **comando.indexOf('B'):** Utiliza el método indexOf() para encontrar la posición de las letras B, Y y F en la cadena del comando. Esto es crucial para poder extraer los números que las siguen.
- **Validación de Formato:** La condición if (baseIdx != -1 && yIdx != -1 && fIdx != -1 && baseIdx < yIdx && yIdx < fIdx) asegura que todas las letras clave estén presentes y en el orden correcto (B antes que Y, Y antes que F). Esto previene errores si se recibe un comando malformado.
- **comando.substring(...) y .toInt():** Estas funciones se usan para extraer las subcadenas que contienen los números (los ángulos y el estado de fuego) y luego convertirlos a valores enteros (toInt()).
- **constrain(value, min, max):** Una función muy útil que fuerza un valor a estar dentro de un rango específico. Si el valor es menor que min, se convierte en min; si es mayor que max, se convierte en max. Esto protege los actuadores de recibir órdenes fuera de sus límites físicos.
- **Lógica de Movimiento de Base:** Solo se llama a moverBase() si el ángulo objetivo de la base difiere del actual en más de 1 grado. Esto evita "temblores" o movimientos innecesarios debido a pequeñas variaciones. Si el cambio es insignificante, se asegura que el motor esté frenado.
- **Lógica de Disparo:** El disparo solo se inicia si la señal de fire es true y si no hay un disparo ya activo (!disparoActivo). Esto evita múltiples activaciones rápidas.
- **enviarConfirmacion(...):** Envía una respuesta al dispositivo emisor del comando, indicando que la orden fue recibida y procesada.
- **Mensaje de Error:** Si el comando no cumple con el formato esperado, y DEBUG_MODE está activo, se imprime un mensaje de error en el Monitor Serial.

2.7 moverBase(): Control de la Rotación Horizontal

Esta función es la responsable de girar la base del lanzador a un ángulo objetivo específico utilizando el motor de corriente continua. La precisión aquí depende de la calibración de TIEMPO_POR_GRADO_....

```
VOID MOVERBASE(INT ANGULOOBJETIVO) {
 INT DIFERENCIA = ANGULOOBJETIVO - POSICIONBASE; // CALCULA LA DIFERENCIA DE GRADOS A MOVER.

 IF(ABS(DIFERENCIA) <= 1) { // SI LA DIFERENCIA ES DE 1 GRADO O MENOS, NO ES SIGNIFICATIVO.
FRENARMOTOR(); // ASEGURA QUE EL MOTOR ESTÉ PARADO.
RETURN; // SALE DE LA FUNCIÓN.
 }

 // --- CONFIGURAR DIRECCIÓN DEL MOTOR ---
 IF (DIFERENCIA > 0) { // SI EL ÁNGULO OBJETIVO ES MAYOR QUE LA POSICIÓN ACTUAL, MOVER EN SENTIDO HORARIO (DERECHA).
DIGITALWRITE(MOTOR_IN1, LOW);
DIGITALWRITE(MOTOR_IN2, HIGH);
 } ELSE { // SI EL ÁNGULO OBJETIVO ES MENOR, MOVER EN SENTIDO ANTIHORARIO (IZQUIERDA).
DIGITALWRITE(MOTOR_IN1, HIGH);
DIGITALWRITE(MOTOR_IN2, LOW);
 }

 // --- CALCULAR EL TIEMPO DE MOVIMIENTO ---
 // MULTIPLICA LA DIFERENCIA ABSOLUTA DE GRADOS POR EL TIEMPO CALIBRADO POR GRADO.
 UNSIGNED LONG TIEMPOMOVIMIENTO = ABS(DIFERENCIA) *
  (DIFERENCIA > 0 ? TIEMPO_POR_GRADO_HORARIO : TIEMPO_POR_GRADO_ANTIHORARIO);
   // USA UN OPERADOR TERNARIO PARA SELECCIONAR EL TIEMPO POR GRADO CORRECTO (HORARIO O ANTIHORARIO).

 IF(DEBUG_MODE) { // MENSAJES DE DEPURACIÓN PARA ESTE MOVIMIENTO.
SERIAL.PRINT("MOVIENDO BASE: ");
SERIAL.PRINT(DIFERENCIA > 0 ? "DERECHA" : "IZQUIERDA");
SERIAL.PRINT(", GRADOS: ");
SERIAL.PRINT(ABS(DIFERENCIA));
SERIAL.PRINT(", TIEMPO: ");
SERIAL.PRINT(TIEMPOMOVIMIENTO);
SERIAL.PRINTLN("MS");
 }

 // --- MOVER MOTOR ---
 ANALOGWRITE(MOTOR_ENA, VELOCIDADMOTOR); // APLICA LA VELOCIDAD PWM AL MOTOR.
 DELAY(TIEMPOMOVIMIENTO); // ESPERA EL TIEMPO CALCULADO. ESTO DETIENE LA EJECUCIÓN DEL PROGRAMA MOMENTÁNEAMENTE.
 FRENARMOTOR(); // DETIENE EL MOTOR DESPUÉS DEL TIEMPO DE MOVIMIENTO.

 // --- ACTUALIZAR POSICIÓN ---
 POSICIONBASE = ANGULOOBJETIVO; // ACTUALIZA LA POSICIÓN ACTUAL DE LA BASE.
}
```

- **int diferencia = anguloObjetivo - posicionBase;:** Calcula cuántos grados debe girar el motor y en qué dirección.
- **int diferencia = anguloObjetivo - posicionBase;:** Calcula cuántos grados debe girar el motor y en qué dirección.
- **Condición de No Movimiento:** Si la diferencia es mínima (<= 1 grado), se asume que la base ya está en posición o muy cerca, y se frena el motor para evitar vibraciones innecesarias.

- **Control de Dirección (digitalWrite(MOTOR_IN1, ...), digitalWrite(MOTOR_IN2, ...)):** Estas líneas controlan las entradas del puente H para definir si el motor gira en sentido horario o antihorario, basándose en si el anguloObjetivo es mayor o menor que posicionBase.
- **unsigned long tiempoMovimiento = ...:** Esta es la parte clave para la precisión. Multiplica la cantidad de grados a mover por el TIEMPO_POR_GRADO_HORARIO o TIEMPO_POR_GRADO_ANTIHORARIO (según la dirección). Este tiempo determina cuánto tiempo el motor estará activo.
- **analogWrite(MOTOR_ENA, velocidadMotor);:** Activa el motor aplicando la señal PWM de velocidad.
- **delay(tiempoMovimiento);:** La función delay() pausa la ejecución del programa por el tiempo calculado. Durante este tiempo, el motor de la base gira.
- frenarMotor();: Después de que el tiempo de movimiento ha transcurrido, se llama a esta función para detener el motor.
- **posicionBase = anguloObjetivo;:** Se actualiza la variable posicionBase para reflejar la nueva posición del lanzador.

2.8 moverServoY(): Control de la Elevación Vertical

Esta función maneja el movimiento del servomotor que ajusta la elevación del cañón.

```
void moverServoY(int angulo) {
 angulo = constrain(angulo, SERVO_Y_MIN, SERVO_Y_MAX); // Limita el ángulo al rango seguro del servo.
 if (servoY.read() != angulo) { // Solo mueve el servo si no está ya en la posición deseada.
 servoY.write(angulo); // Envía la orden al servomotor para moverse al ángulo especificado.
 delay(15); // Pequeña pausa para permitir que el servo alcance su posición.
 if(DEBUG_MODE) {
 Serial.print("Servo Y movido a: ");
 Serial.print(angulo);
 Serial.println(" grados");
 }
 }
}
```

- **angulo = constrain(angulo, SERVO_Y_MIN, SERVO_Y_MAX);:** Asegura que el ángulo solicitado para el servo Y esté dentro de los límites de seguridad definidos (SERVO_Y_MIN y SERVO_Y_MAX). Esto es vital para proteger el servomotor y el mecanismo.
- **if (servoY.read() != angulo):** Esta condición es una optimización. Solo se envía una nueva orden al servo si su posición actual (servoY.read()) es diferente al ángulo deseado. Esto reduce el "jitter" (pequeñas vibraciones) y el consumo de energía si el servo ya está en posición.
- **servoY.write(angulo);:** Esta es la función principal que le dice al servomotor a qué ángulo específico debe moverse.
- **delay(15);:** Un breve delay es necesario después de enviar un comando write() al servo para darle tiempo de moverse físicamente a la nueva posición antes de que el código continúe.

2.9 iniciarDisparo(): El Mecanismo de Activación

Esta función es la encargada de activar el relé que pone en marcha el mecanismo de expulsión del proyectil.

```
void iniciarDisparo() {
 digitalWrite(DISPARO_PIN, HIGH); // Pone el pin de disparo en HIGH para activar el relé.
 tiempoInicioDisparo = millis(); // Guarda el tiempo actual (en ms) para controlar la duración
del pulso.
 disparoActivo = true; // Establece la bandera para indicar que un disparo está en curso.

 if(DEBUG_MODE) {
 Serial.println("DISPARO INICIADO - Activando relé");
 }
}
```

- **digitalWrite(DISPARO_PIN, HIGH);:** Activa el relé conectado al pin DISPARO_PIN. Dependiendo de cómo esté cableado el relé, HIGH puede significar que se energiza y cierra el circuito del mecanismo de disparo.
- **tiempoInicioDisparo = millis();:** Registra el tiempo exacto en milisegundos en el que se inició el pulso de disparo. Esto se utilizará en la función loop() para saber cuándo desactivar el relé.
- **disparoActivo = true;:** Esta bandera se establece en true para indicar que el sistema está en medio de un ciclo de disparo. Esto evita que se inicien nuevos disparos si se recibe otra señal de "fuego" antes de que el pulso actual haya terminado.

2.10 Funciones Auxiliares de Sistema

Estas funciones realizan tareas de apoyo vitales para el funcionamiento y la comunicación del firmware.

frenarMotor(): Detener la Base

```
void frenarMotor() {
 digitalWrite(MOTOR_IN1, LOW); // Pone el pin de dirección 1 en LOW
 digitalWrite(MOTOR_IN2, LOW); // Pone el pin de dirección 2 en LOW
 analogWrite(MOTOR_ENA, 0); // Pone la velocidad (PWM) en 0 para detener
completamente el motor.
}
```

- Esta función apaga el motor de la base al poner ambas señales de dirección en LOW y el PWM de habilitación en 0. Esto detiene el motor de forma efectiva y rápida.

enviarConfirmacion(int baseAngle, int yAngle, bool fire): Feedback de Comando

```
void enviarConfirmacion(int baseAngle, int yAngle, bool fire) {
 Serial.print("OK B"); // Envía el prefijo "OK B"
 Serial.print(posicionBase); // Imprime la posición actual de la base (la que el ESP32 cree tener).
 Serial.print(" Y"); // Prefijo " Y"
 Serial.print(servoY.read()); // Imprime la posición actual leída del servomotor del eje Y.
 Serial.print(" F"); // Prefijo " F"
 Serial.println(fire ? 1 : 0); // Imprime 1 si se solicitó un disparo, 0 si no. Añade un salto de línea al
final.
}
```

- Esta función es la respuesta del ESP32 a un comando recibido. Envía una cadena de texto de vuelta al dispositivo emisor (LaunchassistPy).
- El formato de la respuesta es OK B[pos_base] Y[pos_servo_Y] F[estado_disparo]. Esto permite que el software de control de nivel superior sepa que el comando fue procesado y cuál es el estado actual de los actuadores según el ESP32.
- Es crucial para la retroalimentación y la depuración del sistema completo.

debugEstado(): Monitoreo Detallado (Solo en DEBUG_MODE)

```
void debugEstado() {
 Serial.println("--- ESTADO ACTUAL ---");
 Serial.print("Posición base: ");
 Serial.print(posicionBase);
 Serial.println(" grados");

 Serial print("Servo Y: ");
 Serial.print(servoY.read());
 Serial.println(" grados");

 Serial.print("Relé disparo: ");
 Serial.println(digitalRead(DISPARO_PIN) ? "ACTIVO" : "INACTIVO"); // Muestra si el pin de disparo está HIGH
o LOW.

 Serial.print("Disparo activo: ");
 Serial.println(disparoActivo ? "SI" : "NO"); // Muestra el estado de la bandera interna de disparo.
 Serial.println("--------------------");
}

}
```

- Esta función, invocada periódicamente si DEBUG_MODE es true, imprime un resumen del estado interno del lanzador en el Monitor Serial.
- Muestra la posicionBase actual, el ángulo en el que se encuentra el servoY, el estado directo del pin del DISPARO_PIN (relé), y el estado de la bandera disparoActivo.
- Es una herramienta invaluable para diagnosticar problemas y verificar que el sistema está operando como se espera.

2.11 Funciones de Prueba y Restablecimiento

Estas funciones son utilidades para el diagnóstico y para poner el sistema en un estado conocido.

pruebaRele(): Verificación del Relé

```
void pruebaRele() {
 if(DEBUG_MODE) {
 Serial.println("Iniciando prueba de relé...");
 }

 for(int i = 0; i < 3; i++) { // Realiza la prueba 3 veces.
 digitalWrite(DISPARO_PIN, HIGH); // Activa el relé.
 if(DEBUG_MODE) { Serial.println("Relé ACTIVADO"); }
 delay(300); // Mantiene activo por 300 ms.

 digitalWrite(DISPARO_PIN, LOW); // Desactiva el relé.
 if(DEBUG_MODE) { Serial.println("Relé DESACTIVADO"); }
 delay(300); // Pausa de 300 ms antes de la siguiente iteración.
 }

 if(DEBUG_MODE) {
 Serial.println("Prueba de relé completada");
 }
}
```

- Activada por el comando serial TESTFIRE.
- Realiza un ciclo de activación y desactivación del relé de disparo tres veces con un breve delay entre cada estado. Útil para verificar que el relé y el mecanismo de disparo funcionan físicamente.

resetPosiciones(): Reinicio del Sistema

```
void resetPosiciones() {
 moverBase(90); // Mueve la base a la posición central.
 moverServoY(SERVO_Y_CENTER); // Mueve el servomotor del eje Y a su posición central.
 digitalWrite(DISPARO_PIN, LOW); // Asegura que el relé de d
isparo esté apagado.
 disparoActivo = false; // Reinicia la bandera de disparo.

 if(DEBUG_MODE) {
 Serial.println("Sistema resetado a posiciones iniciales");
 }
}
```

- Activada por el comando serial RESET.
- Lleva el lanzador a un estado inicial y seguro: la base a 90 grados, el servomotor Y a su centro, y el relé de disparo desactivado. Reinicia la bandera disparoActivo. Muy útil para volver a un punto de partida conocido durante las pruebas.

33. Requerimientos del Sistema

3.1 Hardware

- **Microcontrolador:** ESP32 (cualquier modelo compatible con la librería ESP32Servo).
- **Motor DC:** Para el movimiento rotacional de la base.
- **Driver de Motor (Puente H):** Esencial para controlar la dirección y velocidad del motor DC (e.g., L298N).
- **Servomotor:** Para el ajuste de elevación (eje Y) del cañón.
- **Módulo de Relé:** Para activar el mecanismo de disparo del proyectil.
- **Fuente de Alimentación:** Para alimentar el ESP32 y, muy importante, una fuente separada para los motores y el relé si son de alta corriente.

3.2 Software

- **Arduino IDE:** El entorno de desarrollo para cargar el firmware.
- **C++:** El lenguaje de programación utilizado.
- **Librerías de ESP32:** Asegúrate de tener instalada la placa ESP32 en tu Arduino IDE y la librería ESP32Servo.h.

4. Primeros Pasos: Configuración y Carga del Firmware

4.1 Preparación del Entorno (Arduino IDE)

1. Abre el Arduino IDE.
2. Ve a Archivo > Preferencias y en "URLs Adicionales para Gestor de Tarjetas" añade la URL para las placas ESP32:https://raw.githubusercontent.com/espressif/arduino-esp32/gh-pages/package_esp32_index.json
3. Ve a Herramientas > Placa > Gestor de Tarjetas... y busca "esp32", luego instálalo.
4. Ve a Programa > Incluir Librería > Gestionar Librerías... y busca "ESP32Servo", luego instálala.

4.2 Carga del Código al ESP32

1. Abre el archivo autotrigger.ino en el Arduino IDE.
2. Asegúrate de seleccionar tu placa ESP32 (Herramientas > Placa) y el puerto COM correcto (Herramientas > Puerto).
3. Haz clic en "Subir" (la flecha hacia la derecha) para compilar y cargar el código a tu ESP32.

5. Modo de Operación y Comandos

5.1 Conexión Serial

El firmware autotrigger se comunica a través del puerto serial. Para interactuar con él directamente (por ejemplo, para pruebas o depuración), puedes usar el Monitor Serial del Arduino IDE o cualquier otra herramienta de comunicación serial (como minicom en Linux). La velocidad de comunicación debe ser de 115200 baudios.

5.2 Estructura de los Comandos

Los comandos se envían como cadenas de texto con el siguiente formato: BxYyFz
Donde:

- B: Indica el ángulo de la base (movimiento horizontal). x es un valor numérico entre 0 y 180 grados.
- Y: Indica el ángulo del servomotor del eje Y (elevación vertical). y es un valor numérico entre SERVO_Y_MIN y SERVO_Y_MAX (definidos en el código, típicamente 30 a 150 grados).
- F: Indica la acción de disparo. z es 1 para activar el disparo o 0 para no disparar.

Ejemplo de comando: B90Y75F1 Este comando indica al lanzador que se mueva a 90 grados de la base, 75 grados de elevación del servo Y, y que active el disparo.

Cada comando debe finalizar con un carácter de nueva línea (\n).

5.3 Comandos Especiales

Además del formato de comando principal, autotrigger reconoce algunos comandos especiales para facilitar las pruebas y el reinicio:

- TESTFIRE: Al enviar esta cadena (sin el formato BxYyFz), el sistema realizará una prueba de activación del relé de disparo, activándolo y desactivándolo varias veces para verificar su funcionamiento.
- RESET: Al enviar esta cadena, el sistema moverá la base y el servomotor Y a sus posiciones centrales iniciales (base a 90 grados, servo Y a SERVO_Y_CENTER) y asegurará que el relé de disparo esté desactivado.

6. Consideraciones Adicionales

- **Calibración:** Los valores de TIEMPO_POR_GRADO_HORARIO y TIEMPO_POR_GRADO_ANTIHORARIO son críticos. Es posible que necesites ajustarlos mediante pruebas en tu configuración física para lograr la máxima precisión en el movimiento de la base.

- **Seguridad:** Siempre ten precaución al trabajar con sistemas mecatrónicos. Asegúrate de que el relé de disparo esté correctamente cableado y de que no haya proyectiles cargados durante las pruebas o la depuración a menos que sea intencional.

- **Modo Depuración:** Utiliza el DEBUG_MODE (const bool DEBUG_MODE = true;) durante el desarrollo y la solución de problemas. Desactívalo (false) para el funcionamiento normal para evitar sobrecargar el puerto serial con mensajes.

- **Puente H:** Asegúrate de que tu puente H (driver de motor) sea capaz de manejar la corriente requerida por el motor DC de la base. Una fuente de alimentación separada y robusta para los motores es altamente recomendable para evitar problemas de estabilidad en el ESP32.

UNIVERSIDAD AUTÓNOMA DEL CARIBE

MANUAL DE UTILIZACIÓN E IMPLEMENTACIÓN DEL SOFTWARE AUTOTRIGGER

Mateo David Castro Villegas
Geymar David Chung Soto
Duván Cohen Pérez
Carlos Gabriel Díaz Sáenz
Jean Pierre Coll Vásquez
Kelvin De Jesús Beleño Sáenz
Saúl Antonio Pérez Pérez